SAMADHAN.CONNECT

Govt. of Jharkhand Civic Innovation & Problem-Solving Ecosystem

SYSTEM ARCHITECTURE, TECH STACKS & FULL WORKFLOWS

Detailed Operational & Technical Flowchart Blueprint

1. Complete Technology Stack Matrix

Layer	Technologies & Frameworks Used	Architectural Role & Functionality
Frontend Framework	React 19, Vite 8, React Router v7	Ultra-fast SPA rendering, component modularity, client-side routing, optimistic state updates.
Styling & UI	Tailwind CSS v3, PostCSS, Lucide Icons	Modern glassmorphism, responsive grid layouts, high-contrast accessible civic design tokens.
Auth & RBAC	Supabase Auth, OAuth 2.0 (Google, GitHub)	Passwordless secure OAuth login, JWT sessions, first-time role onboarding modal, RBAC.
Database & Persistence	Supabase (PostgreSQL 15), Row Level Security	Relational schema (profiles, problems, solutions, projects, messages), real-time sync.
Backend Gateway	Vercel Serverless (/api/chat.js), Vite Middleware	Zero-exposure environment secrets handling, CORS, rate limiting, and multi-model AI routing.
AI Engine	Google Gemini API (@google/genai, gemini-2.5-pro/flash)	25 strict civic domains, out-of-domain rejection, step-by-step problem-solving guidance.
Micro-Animations	Canvas Confetti (canvas-confetti), Status Badges	Visual milestone rewards and interactive feedback upon problem approval and resolution.

2. How the Backend Works (In Simple Language)

1. Secure Gateway & Key Isolation (Vercel Serverless / Node.js): When a user asks a question or submits a challenge, the frontend calls POST /api/chat. The backend decrypts server environment variables (process.env.GEMINI_API_KEY), guaranteeing that secret credentials never reach the browser.

2. Dual-Engine Resilience (@google/genai + Local Knowledge Base): The backend attempts live reasoning via Google Gemini 2.5 Pro / Flash. If quota or network limits occur, it seamlessly falls back to the **Intelligent Civic Knowledge Engine**, returning verified step-by-step answers without failing.

3. Role-Based Data Security (Supabase PostgreSQL 15 + RLS): Row Level Security policies and database triggers guarantee that citizens only edit their own problems, students collaborate securely, and Nodal Officers sign off on genuine civic projects.

3. Flow Chart 1: Master Civic Problem-Solving Lifecycle (With Tech Stacks)

STEP 1

Citizen Reports Civic Problem — [Citizen]

Tech Stack: React 19 + HTML5 Geolocation + Supabase Storage CDN

Citizen encounters a localized community problem (e.g. broken transformer, water contamination) and submits photos, GPS coordinates, category, and description.

▼ Problem submitted to District Nodal Queue [Postgres DB Trigger]

STEP 2

Triage & Jurisdiction Verification — [Nodal Officer]

Tech Stack: Supabase PostgreSQL 15 + Row Level Security (RLS)

Nodal Officer reviews the authenticity and jurisdiction of the problem. If verified, the issue is formally approved and published as an 'Active Innovation Challenge'.

▼ Challenge published on Academic Innovation Feed [React Router v7]

STEP 3

Solution Proposal & Faculty Mentorship — [Students & Universities]

Tech Stack: React 19 + Markdown Editor + dataService.js

Student teams form cross-disciplinary groups, draft technical blueprints (budget, timeline, prototype design), and obtain Faculty Mentor review before platform submission.

▼ Solution validated & matched with CSR sponsors [Realtime WebSockets]

STEP 4

CSR Sponsorship & Milestone Funding — [Industry & CSR Partners]

Tech Stack: PostgreSQL Ledger Schema + Analytics Dashboard

Corporate partners evaluate project feasibility and community impact ROI. Pledges CSR funding, grants, or equipment disbursed on milestone completion.

▼ Funds disbursed for ground prototype deployment

STEP 5

Ground Deployment, Field Audit & Sign-off — [Nodal Officer & Citizen]

Tech Stack: Canvas Confetti + Verified Badge + Notification Trigger

Students deploy the prototype on-site. The Nodal Officer conducts a physical inspection, and the reporting Citizen confirms resolution with a 5-star satisfaction rating.

4. Flow Chart 2: System Architecture Data Pipeline (With Tech Stacks)

```
+---------------------------------------------------------------------------------------------------+ | 1. CLIENT TIER (React
19 / Vite 8 SPA / Tailwind CSS v3) | | [Citizen Portal] [Student Lab] [University Hub] [Industry CSR] [Nodal Admin Desk] | | •
Tech: React Router v7, Lucide Icons, Canvas Confetti, MarkdownMessage, SuggestedDomainsBar |
+---------------------------------------------------------------------------------------------------+ ■ ■ REST / Supabase JS
API (Auth/CRUD) POST /api/chat (AI Stream) ■ ■ ▼ ▼ +-----------------------------------------------------+
+------------------------------------------+ | 2. DATABASE & AUTH TIER (Supabase) | | 3. BACKEND / API GATEWAY TIER | | •
PostgreSQL 15 Relational DB (RLS Policies) | | • Vercel Serverless (Node.js api/chat.js) | • Supabase Auth (JWT & OAuth
Google/GitHub) | | • Vite Dev Server Custom Middleware | | • Database Triggers & Auto-Profile Sync | | • Server Config
(chatbotPrompt.js) | | • Tables: profiles, problems, solutions, messages | | • Server Secret (process.env.GEMINI_KEY)|
+-----------------------------------------------------+ +------------------------------------------+ ■ @google/genai SDK Call
(gemini-2.5-pro) ■ ▼ +------------------------------------------+ | 4. EXTERNAL AI CLOUD TIER | | • Google Gemini 2.5 Pro /
Flash Models | | • 25 Strict Civic Domain Guardrails | | • Step-by-Step Problem Solving Logic |
+------------------------------------------+
```

5. Flow Chart 3: Authentication & Role Onboarding (With Tech Stacks)

```
[ User clicks 'Sign in with Google' or 'GitHub' ] <■■■ [ Tech: HeroLoginCard.jsx / Navbar.jsx ] ■ ▼ [ Supabase OAuth handles
Authentication & generates JWT Token ] <■■■ [ Tech: supabase.auth.signInWithOAuth() ] ■ ▼ [ PostgreSQL Database Trigger
checks 'profiles' table ] <■■■ [ Tech: on_auth_user_created trigger function ] ■■■ Existing Profile ■■> [ Load Profile &
Redirect to Role Workspace ] ■■■ New Profile ■■■■■■■■■> [ Insert Basic Profile with Email & Name ] ■ ▼ [ Frontend detects
missing role & opens Modal ] <■■■ [ Tech: RoleSelectionModal.jsx (React 19 Portal) ] [ User selects 1 of 5 Roles: Citizen |
Student | University | Industry | Nodal Officer ] ■ ▼ [ DataService updates profile.role in DB ] <■■■ [ Tech:
dataService.updateProfile() ] ■ ▼ [ Special Condition: Is email == 'microsoft1gab@gmail.com'? ] <■■■ [ Tech: Super Admin
Privilege Check ] ■■■ YES ■■> [ Grant SUPER_ADMIN role with full /admin panel access to promote users ] ■■■ NO ■■> [ Grant
Standard Verified Role Workspace Dashboard ]
```

6. Flow Chart 4: Samadhan AI 25-Domain Decision Engine (With Tech Stacks)

```
[ User sends message or clicks Chip ] <■■■ [ Tech: MessagingPage.jsx + SuggestedDomainsBar.jsx ] ■ ▼ [ Frontend sanitizes
input & slices history to last 10 messages ] <■■■ [ Tech: aiChatService.js ] ■ ▼ [ POST /api/chat JSON Payload ] <■■■ [
Tech: Fetch API HTTP Stream ] ■ ▼ [ Backend Gateway checks process.env.GEMINI_API_KEY ] <■■■ [ Tech: Vercel Serverless /
Node.js Runtime ] ■■■ API Key Present ■■> [ Initialize Google Gen AI SDK ] <■■■ [ Tech: @google/genai SDK v2.20.0 ] ■ ■ ■ ▼
■ [ Call Gemini 2.5 Pro / Flash ] <■■■ [ Tech: ai.models.generateContent() ] ■ ■■■ Success ■■> [ Return JSON reply & status:
success ] ■ ■■■ 404/429/Timeout ■■> [ Failover to Local Knowledge Engine ] ■ ■■■ No API Key ■■■■■■■■■> [ Run Intelligent
Local Knowledge Engine ] <■■■ [ Tech: 25-Domain Heuristic Engine ] ■■■ Is In-Domain? ■■■■> [ Return Step-by-Step Civic Steps
] ■■■ Is Out-of-Domain? ■> [ Return Official Canned Refusal ] ■ ▼ [ Frontend receives JSON reply & renders UI ] <■■■ [ Tech:
MarkdownMessage.jsx (RegExp Tokenizer) ]
```

Specification Summary: *This document provides the complete frontend, backend, database, and AI technology stack flowcharts for Samadhan.Connect. Engineered for state-wide citizen empowerment, robust security, and seamless civic collaboration.*